

DevOps CI/CD Pipeline using Jenkins, Docker, and Kubernetes

Project Objective

Design and automate a robust CI/CD pipeline to deploy a multi-tier web application (frontend and backend) using:

- Jenkins for Continuous Integration and Continuous Deployment
- Docker for containerization
- Kubernetes for orchestration (via Minikube)
- AWS EC2 (c7i-flex.large) as the host environment
- GitHub as the code repository

The goal is to build a reliable, production-ready DevOps workflow.

Hosting Environment

- Cloud Platform: AWS EC2
- Instance Type: c7i-flex.large (2 vCPU, 4 GB RAM)
- Operating System: Ubuntu 24.04 LTS
- Security Group Ports Opened: 22 (SSH), 8080 (Jenkins), 30080 (Backend), 30081 (Frontend)

- Minikube Tunnel: Enabled using minikube tunnel
--bind-address 0.0.0.0

Tools & Technologies

Tool	Purpose
GitHub	Version Control
Docker	Containerize frontend/backend apps
Jenkins	Automate CI/CD pipeline
Kubernetes	Manage containerized workloads
Minikube	Local Kubernetes cluster
Nginx	Frontend static file server
Flask	Python backend web server

Project Structure

DevOps-CI-CD-Pipeline-Using-Jenkins-Docker-and
-Kubernetes/

- |— backend/
 - | |— Dockerfile
 - | |— app.py
 - | |— requirements.txt
- |— frontend/
 - | |— Dockerfile
 - | |— index.html
- |— k8s/
 - | |— backend-deployment.yaml
 - | |— frontend-deployment.yaml
 - | |— service.yaml
- |— Jenkinsfile
- |— docker-compose.yml
- |— [README.md](#)

Setup and Installation

Install Required Tools

→sudo apt update && sudo apt install -y git curl [docker.io](https://docs.docker.io)

→sudo usermod -aG docker \$USER

Install Minikube:

→curl -LO

<https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64>

→sudo install minikube-linux-amd64
/usr/local/bin/minikube

Install kubectl:

→curl -LO "https://dl.k8s.io/release/\$(curl -L -s
<https://dl.k8s.io/release/stable.txt>)/bin/linux/amd64/kubectl"

→chmod +x kubectl && sudo mv kubectl /usr/local/bin/

Start Minikube:

→minikube start --driver=docker

→minikube tunnel --bind-address 0.0.0.0

Docker: Containerization

Backend Dockerfile

```
FROM python:3.9-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install Flask==2.0.3 Werkzeug==2.0.3
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

Frontend Dockerfile

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Build & Push Images

```
→docker build -t shivakumar46/devops-backend:latest
./backend
```

```
→docker build -t shivakumar46/devops-frontend:latest
./frontend
```

```
→docker push shivakumar46/devops-backend:latest
```

```
→docker push shivakumar46/devops-frontend:latest
```

Kubernetes Deployment

Backend Deployment

- Kind: Deployment
- Name: devops-backend
- Replicas: 2
- Container Image: tpathak21/devops-backend:latest
- Container Port: 5000
- Label: app: devops-backend

Purpose: Launches two backend container replicas managed by Kubernetes

Backend Service

- Kind: Service
- Name: devops-backend-service
- Type: NodePort
- Selector: app: devops-backend
- Ports:
 - Port: 5000
 - TargetPort: 5000
 - NodePort: 30080

Purpose: Exposes backend app outside the cluster on EC2 instance's IP:30080

Frontend Deployment

- Kind: Deployment
- Name: devops-frontend
- Replicas: 2
- Container Image: shivakumar46/devops-frontend:latest
- Container Port: 80
- Label: app: devops-frontend
- Purpose: Launches two frontend container replicas

Frontend Service

- Kind: Service
- Name: devops-frontend-service
- Type: NodePort
- Selector: app: devops-frontend
- Ports:
 - Port: 80
 - TargetPort: 80
 - NodePort: 30081

Purpose: Exposes frontend app outside the cluster on EC2 instance's IP:30081

Apply Resources

- `kubectl apply -f k8s/`
- `kubectl get pods -o wide`
- `kubectl get svc`

Jenkins CI/CD Pipeline

Agent Declaration

- Type: any
- Purpose: Allows pipeline to run on any available Jenkins agent (node)

Environment Setup

- DOCKERHUB_CREDENTIALS:
- Secure Jenkins credentials (ID: dockerhub-creds) used for DockerHub login

Stages

Stage 1: Checkout Code from GitHub Clones the main branch from your GitHub repository:

<https://github.com/shivakumarshivas/Devops-Project.git>

Stage 2: Build Backend Docker Image

- Builds Docker image:
 - Image: shivakumar46/devops-backend:latest
 - Context: ./backend

Stage 3: Build Frontend Docker Image

- Builds Docker image:
 - Image: shivakumar46/devops-frontend:latest
 - Context: ./frontend

Stage 4: Login to DockerHub

- Authenticates Jenkins with DockerHub using securely stored credentials

Stage 5: Push Docker Images

- Pushes both frontend and backend images to DockerHub:
 - shivakumar46/devops-backend:latest
 - shivakumar46/devops-frontend:latest

Stage 6: Deploy to Kubernetes

- Uses kubectl to apply manifests for:
 - Backend Deployment
 - Frontend Deployment
 - NodePort Services
- Uses the Kubeconfig file located at:
/var/lib/jenkins/.kube/config

End Result

- Fully automated CI/CD pipeline from GitHub to DockerHub to a live Kubernetes cluster
- Runs seamlessly inside an AWS EC2 instance with Jenkins managing every stage

Output Validation

curl Test

- curl http://<EC2-PUBLIC-IP>:30080 # Backend
- curl http://<EC2-PUBLIC-IP>:30081 # Frontend

Jenkins Pipeline

All stages: Clone > Build > Push > Deploy should show as green

Outputs Images

The screenshot shows an AWS CloudShell terminal window with the following content:

```
1 additional security update can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

Last login: Tue Jun 23 12:02:41 2026 from 13.233.177.4
ubuntu@ip-172-31-4-116:~$ kubectl get pods
kubectl get svc
minikube service list
```

NAME	READY	STATUS	RESTARTS	AGE
devops-backend-7f987b476d-j7qmn	1/1	Running	1 (33m ago)	23h
devops-backend-7f987b476d-r6krr	1/1	Running	1 (33m ago)	23h
devops-frontend-7598c985fc-298lk	1/1	Running	1 (22h ago)	23h
devops-frontend-7598c985fc-zc6td	1/1	Running	1 (22h ago)	23h

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
devops-backend-service	NodePort	10.100.237.187	<none>	5000:30080/TCP	23h
devops-frontend-service	NodePort	10.108.239.172	<none>	80:30081/TCP	23h
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	3d23h

NAMESPACE	NAME	TARGET PORT	URL
default	devops-backend-service	5000	http://192.168.49.2:30080
default	devops-frontend-service	80	http://192.168.49.2:30081
default	kubernetes	No node port	
kube-system	kube-dns	No node port	

```
ubuntu@ip-172-31-4-116:~$
```

i-0a4b12d323830d50c (Shiva)

PublicIPs: 13.201.77.126 PrivateIPs: 172.31.4.116

CloudShell Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Hot weather Now

Search

05:42 PM 23-06-2026

Jenkins / Pipeline project / #3

Status

Changes

Console Output

Edit Build Information

Delete build '#3'

Timings

Git Build Data

Pipeline Overview

Open Blue Ocean

Restart from Stage

Replay

Pipeline Steps

Workspaces

Previous Build

Console

Download Copy View as plain text

```
Started by user Shiva
Obtained Jenkinsfile from git https://github.com/shivakumarshivas/Devops-Project.git
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/Pipeline project
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
using credential github-cred
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/Pipeline project/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/shivakumarshivas/Devops-Project.git # timeout=10
Fetching upstream changes from https://github.com/shivakumarshivas/Devops-Project.git
> git --version # timeout=10
> git --version # 'git version 2.43.0'
using GIT_ASKPASS to set credentials github
> git fetch --tags --force --progress -- https://github.com/shivakumarshivas/Devops-Project.git
+refs/heads/*:refs/remotes/origin/* # timeout=10
```

Feels hotter Now

Search

ENG IN

05:45 PM 23-06-2026

AWS Manager | SecurityGroup | EC2 Instance Cor | EC2 Instance Cor | EC2 Instance Cor | EC2 Instance Cor | +

ap-south-1.console.aws.amazon.com/ec2/home?region=ap-south-1#SecurityGroup:group-id=sg-06537a22bb83b999c

aws [Alt+S] Asia Pacific (Mumbai) Shiva k (109734929971) Shiva k

EC2 > Security Groups > sg-06537a22bb83b999c - default

EC2

Dashboard

AWS Global View

Events

Instances

Instances

Instance Types

Launch Templates

Spot Requests

Savings Plans

Reserved Instances

Dedicated Hosts

Capacity Reservations

Capacity Manager New

Images

AMIs

109734929971 / Permission entries 1 Permission entry

Inbound rules Outbound rules Sharing VPC associations Related resources - new Tags

Inbound rules (7)

Search

Manage tags Edit inbound rules

Type	Protocol	Port range	Source	Description
SSH	TCP	22	0.0.0.0/0	-
Custom TCP	TCP	30081	0.0.0.0/0	-
All traffic	All	All	sg-06537a22bb83b999...	-
Custom TCP	TCP	8080	0.0.0.0/0	-
HTTP	TCP	80	0.0.0.0/0	-
Custom TCP	TCP	30080	0.0.0.0/0	-
HTTPS	TCP	443	0.0.0.0/0	-

CloudShell Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Jenkins / Pipeline project / #3

Browser tabs: AWS M, Security, EC2 Ins, EC2 Ins, EC2 Ins, EC2 Ins, DevOp, 13.201, shivaku, Ask Gemini

Address bar: github.com/shivakumarshivas/Devops-Project

Navigation: Code, Issues, Pull requests, Agents, Actions, Projects, Wiki, Security and quality, Insights, Settings

Devops-Project

Public

main 1 Branch 0 Tags

Go to file Add file Code

File	Commit	Time
backend	Update Dockerfile	last year
frontend	final-update index.html	last year
k8s	firstcommit	yesterday
Jenkinsfile	firstcommit	yesterday
README.md	Update README.md	last year
docker-compose.yml	Initial commit	last year

29 Commits

README

https://github.com/users/shivakumarshivas/packages?repo_name=Devops-Project

Hot weather Now

Search

Windows Taskbar: File Explorer, Edge, Chrome, etc.

System Tray: ENG IN, 05:43 PM 23-06-2026

Browser tabs: AWS Management Con, SecurityGroup | EC2, EC2 Instance Connect, EC2 Instance Connect, DevOps CI/CD Pipeline, Ask Gemini

Address bar: 13.201.77.126:30081

CI/CD Pipeline Project

Automated Deployment using Jenkins, Docker & Kubernetes

Project Overview

This project demonstrates a fully automated CI/CD pipeline built with Jenkins, Docker, and Kubernetes running on an AWS EC2 instance.

Tech Stack

Jenkins Docker Kubernetes Minikube AWS EC2 GitHub HTML Flask

Live Endpoints

- Frontend: <http://YOUR-EC2-IP:30081>
- Backend: <http://YOUR-EC2-IP:30080>

Feels hotter Now

Search

Windows Taskbar: File Explorer, Edge, Chrome, etc.

System Tray: ENG IN, 05:31 PM 23-06-2026

hub Explore My Hub Search Docker Hub CtrlK

shivakumar46 Your personal account

Repositories

Hardened Images

Collaborations

Settings

Default privacy

Notifications

Billing

Usage

Pulls

Storage

Upgrade plan

Repositories

All repositories within the shivakumar46 namespace.

Search by repository name All content Create a repository

Name	Last Pushed	Contains	Visibility	Scout
shivakumar46/devops-backend	30 minutes ago	IMAGE	Public	Inactive
shivakumar46/devops-frontend	31 minutes ago	IMAGE	Public	Inactive
shivakumar46/leo-bakery	4 months ago	IMAGE	Public	Inactive

1-3 of 3

32°C Partly sunny 06:41 PM 22-06-2026

Jenkins Pipeline project #3 Stages

Started by Shiva Started 10 sec ago Queued 3 ms Build has been executing for 10 sec

Start End

Checkout SCM Checkout Code from GitHub Build Backend Image Build Frontend Image Login to DockerHub Push Images Deploy to Kubernetes

Push Images 6s Started 7s ago Jenkins

Checkout SCM 0.67s

Checkout Code from GitHub 0.71s

Build Backend Image 0.62s

Build Frontend Image 0.49s

Checks if running on a Unix-like node

docker tag "\$JD_ID" "\$JD_TAGGED_IMAGE_NAME"

Checks if running on a Unix-like node

24ms 20ms

Light rain At night 06:35 PM 22-06-2026

Links

GitHub Repository:

<https://github.com/shivakumarshivas/Devops-Project.git>

LinkedIn Profile:

<https://www.linkedin.com/in/shiva-kumar11/>

DockerHub Profile:

<https://hub.docker.com/repositories/shivakumar46>

Conclusion

This project is a full-fledged demonstration of how to containerize applications, automate deployments using Jenkins, and orchestrate containers via Kubernetes – all deployed on an AWS EC2 instance. It proves your ability to integrate multiple DevOps tools in a production-like setup.